입출금 정리표 앱의 아키텍처 및 데이터 흐름

app.py·finance.py·sample_transactions.csv 로 이해하는 Streamlit 분석 앱

CodeFiles : app.py, finance.py, sample_transactions.csv



1. 개요와 학습목표

1.1 학습 목표

- 화면·상태 관리와 데이터 처리 확인
- 원본 데이터, 파생 데이터, 필터 결과, 집계 결과를 구분
- 입력 데이터가 검증을 거쳐 확정되는 과정을 추적
- Streamlit 재실행과 세션 상태의 관계 확인.
- 화면과 CSV 에 적용되는 필터 범위를 비교.

- 검증 오류, 빈 데이터, 수입 0 원 등의 예외 상황 확인

1.2 진행 순서

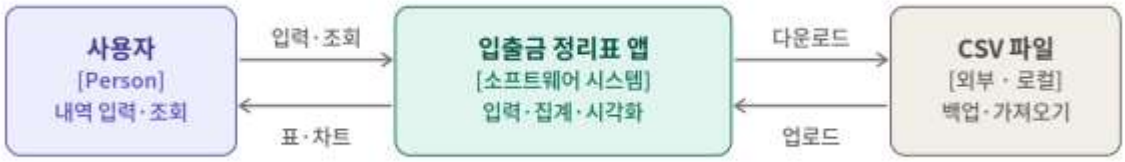
순서	주제	학습 핵심
1	시스템 전체 구조	브라우저와 서버, 파일과 메모리 구분
2	두 파일의 책임	app.py 의 조정 역할과 finance.py 의 계산 역할
3	데이터 모델	4 개 원본 열에서 파생열·집계표로의 변화
4	조회 데이터 흐름	기간 선택에서 KPI·차트·CSV 까지 추적
5	입력 확정 흐름	검증 성공 시에만 상태 갱신
6	실습과 확인	기준값 변경·오류 입력·다운로드 비교

“이 값은 어디에서 들어왔고, 어떤 함수를 거쳐, 어디에 보관되며, 어느 화면과 파일에 사용되는가?” 확인

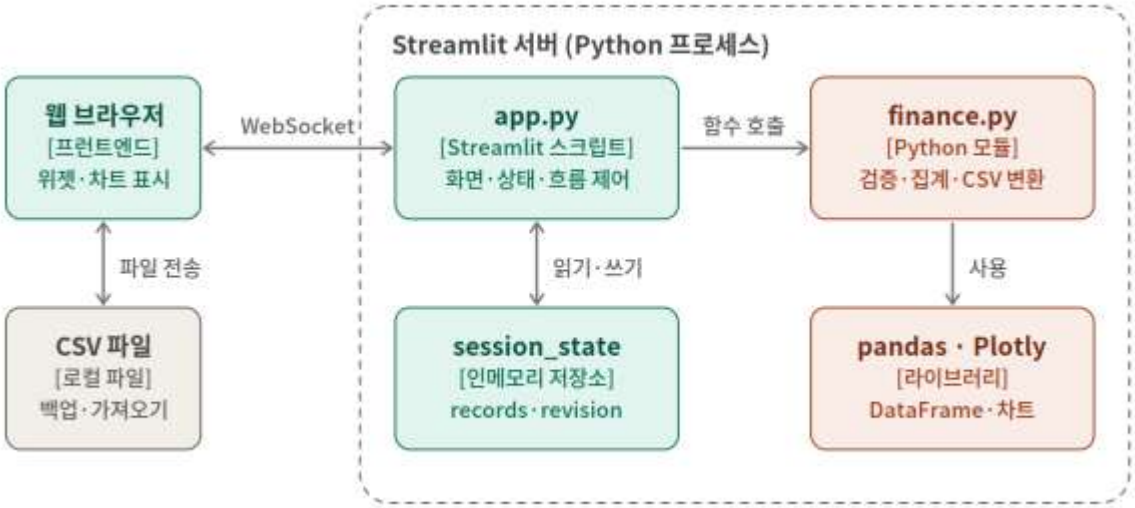
---

2. 시스템 아키텍처

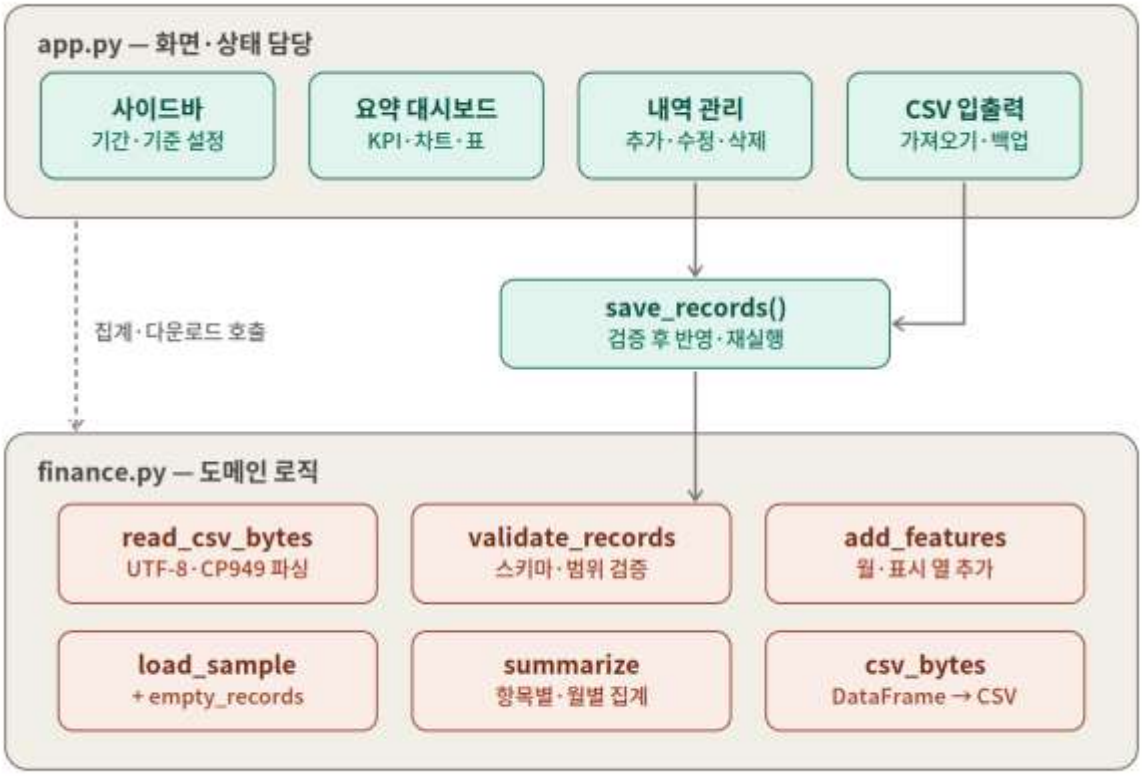
= context



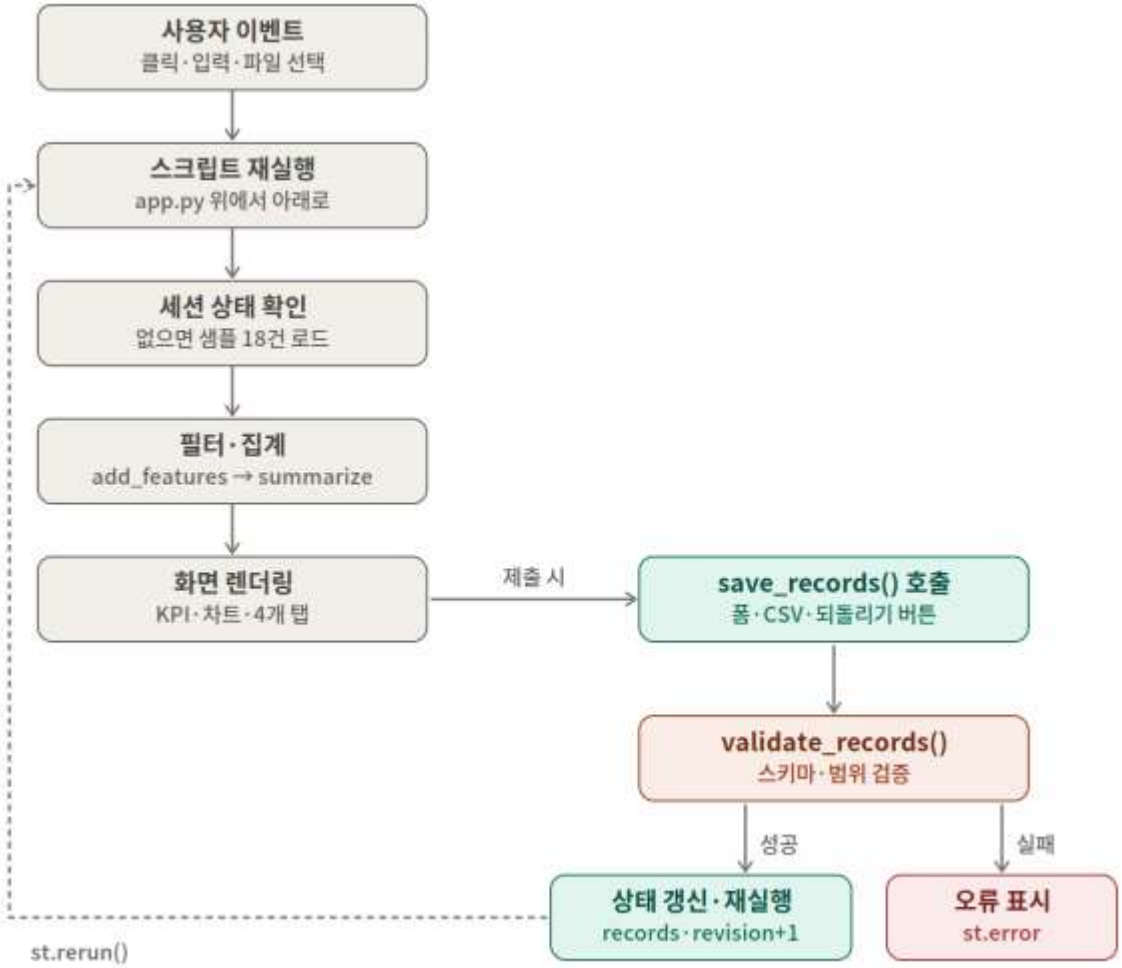
= container

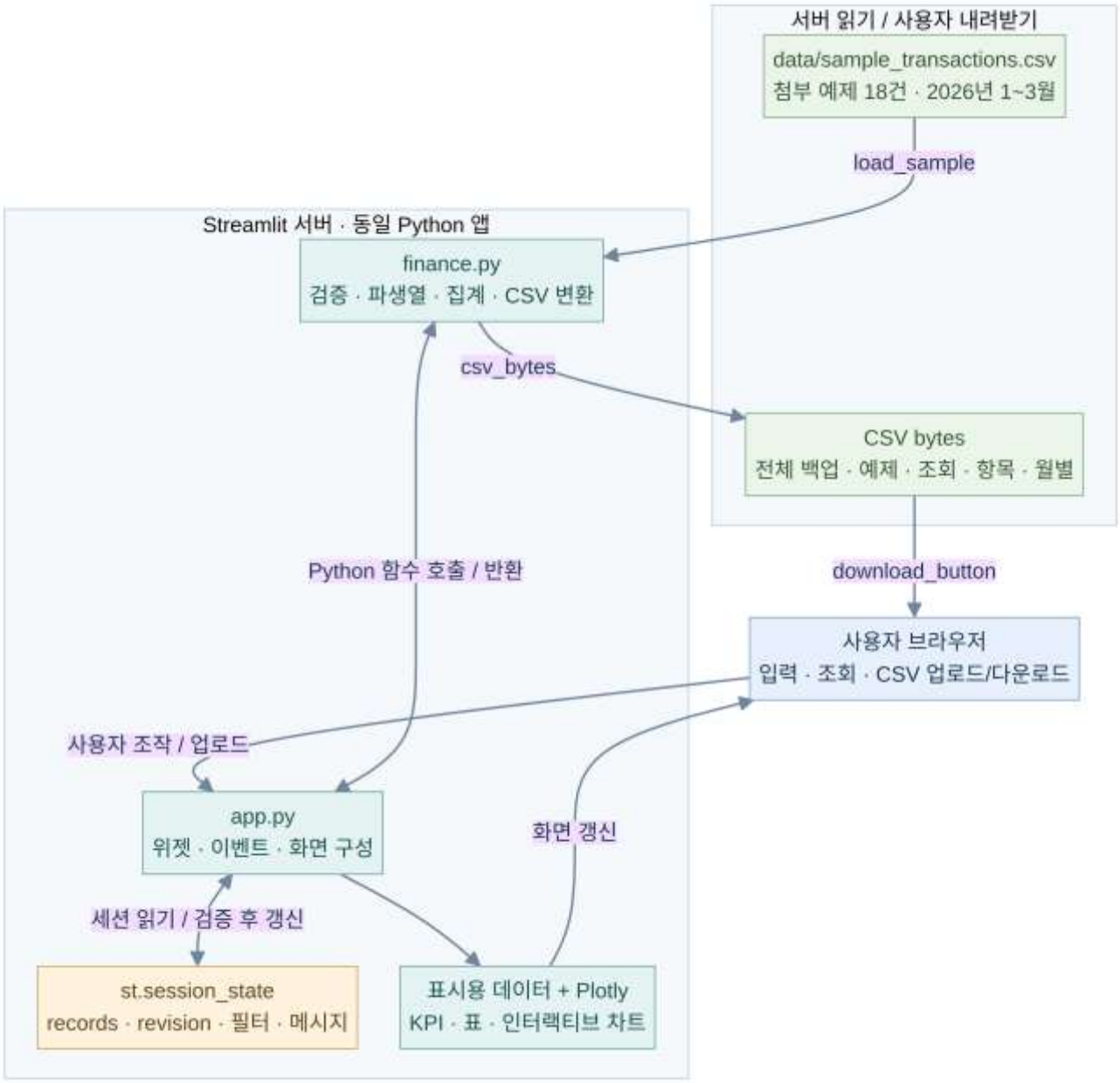


= component



= flow


사용자 이벤트
클릭·입력·파일 선택
스크립트 재실행
app.py 위에서 아래로
세션 상태 확인
없으면 샘플 18건 로드
필터·집계
add_features → summarize
화면 렌더링
KPI·차트·4개 탭
제출 시
save_records() 호출
폼·CSV·되돌리기 버튼
validate_records()
스키마·범위 검증
성공
실패
상태 갱신·재실행
records·revision+1
오류 표시
st.error
st.rerun()



2.1 구조의 핵심

하나의 Streamlit 서버 앱 안에서 화면 구성과 데이터 계산을 함께 수행하는 구조임.
app.py 와 finance.py 를 파일로 분리했지만, 별도의 프런트엔드 서버·백엔드 API 서버로 분리한 것은 아님.

구성 요소	위치	책임
사용자 브라우저	사용자 측	위젯 조작, 표·차트 확인, CSV 업로드·다운로드
app.py	서버 Python	화면 구성, 사용자 입력 처리, 세션 갱신, 데이터 함수 호출
finance.py	서버 Python	입력 검증, 데이터 정규화, 파생열 생성, 집계, CSV 변환
st.session_state	서버 측 사용자 세션 메모리	확정 내역과 위젯 관련 상태 보관
pandas / NumPy	서버 Python	표 연산, 금액·날짜 검증, 결측값 처리
Plotly	서버에서 차트 정의 생성, 브라우저에서 표시	항목별·월별 인터랙티브 그래프
예제 CSV	서버 파일	최초 예제 로딩과 예제 복원·다운로드에 사용
다운로드 CSV	메모리에서 바이트 생성 후 사용자에게 전달	사용자 측 파일 저장

구현되지 않은 영역: 영구 저장 DB, 별도 REST API, 사용자 인증, 은행 계좌 연동이 없음. 이 코드 자체에는 사용자 거래를 서버 디스크에 저장하는 로직 없음.

2.2 두 파일의 역할 분리

app.py: 사용자와 데이터 처리를 연결

화면을 만들고 사용자의 액션이 일어남. finance.py 의 함수를 호출해 결과를 받아 KPI·표·차트·다운로드 버튼에 연결. 확정 데이터를 세션에 반영하는 save_records() 정의

finance.py: 화면 없이 데이터 처리를 수행하도록 분리

pandas DataFrame 이나 bytes 를 받아 DataFrame·집계표·bytes 를 반환하도록 구성.
따라서 데이터 처리 함수는 웹 화면을 띄우지 않고도 실행·검사할 수 있음.

중요한 구분: 모든 계산을 finance.py 로 옮긴 것은 아님. 기간 필터, 전체 수입·지출 KPI, 큰 지출 표 선택, 목표와의 차이, 차트용는 app.py 에 남아 있음.

2.3 파일 구성과 실행 전제

```
프로젝트/
├── app.py
├── finance.py
└── data/
    └── sample_transactions.csv    # 첨부 예제 18건 포함
```

`= finance.py` — 데이터 처리 함수

함수	입력	반환값	주요 역할
`empty_records()`	없음	빈 `DataFrame`	날짜·구분·항목·금액 4개 열과 기본 자료형을 갖춘 빈 표를 생성함.
`validate_records(frame)`	거래 `DataFrame`	검증·정규화된 `DataFrame`	필수 열, 행 수, 날짜, 구분, 항목, 금액을 검사함. 잘못된 데이터는 `ValueError`로 반영을 막음.
`load_sample()`	없음	검증된 예제 `DataFrame`	`data/sample_transactions.csv`를 읽고 검증함.
`read_csv_bytes(content)`	CSV 파일의 `bytes`	검증된 `DataFrame`	파일 크기·빈 파일·인코딩·CSV 형식을 확인하고 거래 데이터를 검증함.
`add_features(frame, threshold=100000)`	거래 표, 큰 지출 기준	파생열을 추가한 `DataFrame`	`월`과 `표시` 열을 생성함. 수입·큰 지출·일반 지출로 분류함.
`summarize(frame)`	`월` 열이 포함된 거래	`(categories, monthly)`	항목별 지출 합계와 월별 수입·지출·남은 금액·남은비율을 계산함.

함수	입력	반환값	주요 역할
	표		
`csv_bytes(frame)`	내보낼 `DataFrame`	CSV `bytes`	UTF-8 BOM을 포함한 CSV로 변환함. 문자 셀의 수식 실행 위험을 완화함.

호출 관계

text

```
load_sample()
└ validate_records()
   └ 빈 표이면 empty_records()

read_csv_bytes()
└ validate_records()

검증된 거래 표
└ add_features()
   └ summarize()

내보낼 표
└ csv_bytes()
```

주의사항: `summarize()`는 `월` 열을 직접 생성하지 않음. 따라서 기본 거래 표에 `add_features()`를 먼저 적용해야 함.

= app.py — 화면·세션 관리 함수

함수	입력	반환형	주요 역할
`save_records(frame, message)`	반영할 전체 거래 표, 성공 메시지	`None`	검증 성공 후 세션 원본을 갱신하고, 편집기 버전 증가·필터 초기화 요청·메시지 설정 후 앱을 재실행함.

내부 처리 순서

text

```
save_records(frame, message)
├ validate_records(frame)
├ session_state.records = clean
├ session_state.revision += 1
├ session_state.reset_filters = True
├ session_state.message = message
└ st.rerun()
```

검증에 실패하면 세션 원본을 갱신하지 않음. 함수 이름은 `save_records()`이지만 **파일·DB 저장이 아니라 현재 사용자 세션의 데이터를 반영하는 함수**

* `finance.py` 에서 가져와 사용하는 함수

`empty_records()`, `validate_records()`, `load_sample()`, `read_csv_bytes()`, `add_features()`, `summarize()`, `csv_bytes()`를 가져와 사용했음.

그 외 화면 구성과 이벤트 처리는 별도의 `main()` 함수 없이 **파일 최상위 코드에서 실행되도록 구성**

. `st.dataframe()`, `st.download_button()`, `px.bar()`, `pd.concat()` 등은 직접 정의한 함수가 아닌 외부 라이브러리 기능임.

= `sample_transactions.csv` — 예제 데이터 파일

함수 정의가 없는 데이터 파일임. 날짜·구분·항목·금액 4개 열과 거래 18건을 담았으며, `load_sample()` 또는 `read_csv_bytes()`의 입력으로 사용

load_sample()은 finance.py 위치를 기준으로 data/sample_transactions.csv 를 읽도록 구현. 추가 첨부된 CSV 를 ZIP 의 프로젝트/data/에 배치. finance.py 옆에 CSV 를 놓는 것만으로는 이 경로와 일치하지 않으므로 data 하위 폴더가 필요함. 실행 위치와 관계없이 Path(__file__).parent 를 기준으로 예제 파일을 찾음.

= 패키시 설치 및 실행

```
python -m pip install streamlit pandas numpy plotly
```

```
streamlit run app.py
```

---

3. 핵심 데이터 모델

3.1 원본 거래 표: records

열	정규화 후 형태	허용 규칙
날짜	pandas string	실제 날짜, YYYY-MM-DD, 1900~2100 년
구분	pandas string	수입 또는 지출
항목	pandas string	앞뒤 공백 제거 후 1~40 글자
금액	int64	1 원~1 조 원의 양의 정수

금액의 부호가 아니라 구분으로 수입과 지출을 나누었음. 따라서 음수 환불·이체·카드대금 중복 등은 현재 모델에서 별도의 정리 규칙이 필요한 영역임.

validate_records()는 필수 4 개 열만 남긴 새 표를 반환함. 추가 열은 제거하며 행 인덱스는 0 부터 다시 매김. 원본 DataFrame 을 직접 수정하지 않음.

3.2 데이터가 변하는 단계

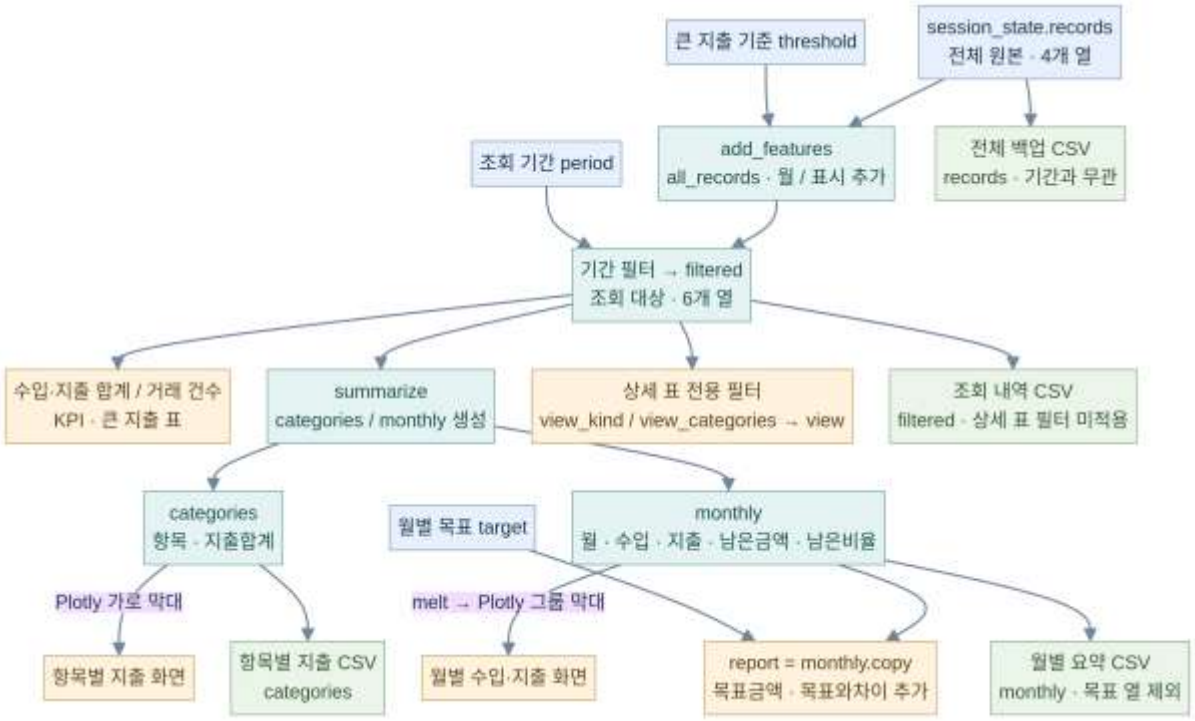
변수	생성 위치	내용	사용 목적
st.session_state.records	app.py	검증된 전체 거래, 기본 4 열	확정 원본·전체 편집·전체 백업
all_records	add_features() 호출	전체 거래 + 월, 표시	파생열이 있는 조회 기반
filtered	app.py	조회 기간을 적용한 거래, 6 열	KPI·차트·집계·기간 기준 CSV
categories	summarize()	항목, 지출합계	지출 순위·가로 막대·항목별 CSV

변수	생성 위치	내용	사용 목적
monthly	summarize()	월별 수입·지출·남은금액·비율	월별 차트·요약·월별 CSV
report	app.py	monthly 복사본 + 목표 관련 2 열	화면의 월별 목표 비교
view	app.py	filtered 에 구분·항목 조건 적용	내역 관리의 상세 조회 표만 표시
candidate	read_csv_bytes()	업로드 CSV 를 검증한 기본 4 열	적용 전 미리보기

핵심: 원본에 모든 열을 계속 추가하는 방식이 아님. 원본을 복사하거나 집계한 별도 표를 단계별로 만듬 . report 를 수정해도 monthly 에 목표 열이 자동 추가되지 않음.

---

4. 조회·분석 데이터 흐름



4.1 전체 처리 경로

확정 원본 → 파생열 생성 → 기간 필터 → 집계 → KPI·표·차트·다운로드 순서로 처리.

```
all_records = add_features(st.session_state.records, int(threshold))
```

```
filtered = (
    all_records.copy()
    if period == "전체 기간"
    else all_records[all_records["월"] == period].copy()
)
```

```
categories, monthly = summarize(filtered)
```

4.2 1 단계: 파생열 생성

finance.py 의 add_features(frame, threshold)에서 두 열을 추가.

- 월: 날짜 문자열의 앞 7 글자를 추출.
- 표시: 수입이면 수입, 지출이면 기준 금액에 따라 큰 지출 또는 일반 지출로 분류.

```
result["월"] = result["날짜"].str[:7]
```

```
result["표시"] = result.apply(
    lambda row: "수입" if row["구분"] == "수입"
    else ("큰 지출" if row["금액"] >= threshold else "일반 지출"),
    axis=1,
)
```

위 예시는 데이터가 있는 경우의 핵심 부분을 발췌. 원본은 빈 표일 때 빈 string Series 를 넣는 분기도 구현.

기존 노트북과의 차이: 금액 열 하나에 함수를 적용한 Series.apply()가 아니라, axis=1 로 **행 단위** 적용을 수행. 한 행의 구분과 금액을 함께 읽기 때문임.

4.3 2 단계: 조회 기간 적용

사이드바의 period 가 전체 기간이면 전체 데이터를 복사함. 특정 월이면 해당 월만 선택함. 이 결과인 filtered 를 여러 화면과 다운로드가 공통으로 사용

이 시점에는 view_kind 와 view_categories 를 아직 적용하지 않았음. 따라서 상세 조회 표에서 '쇼핑'만 선택해도 전체 대시보드가 쇼핑만 집계하는 것은 아님.

4.4 3 단계: KPI 와 항목별 집계

KPI 는 app.py 에서 filtered 의 수입·지출 금액을 합산해 계산. 조회 건수는 len(filtered)

항목별 집계는 finance.py 에서 지출만 선택한 뒤 수행.

```
expenses = frame[frame["구분"] == "지출"]
```

```
categories = (
    expenses.groupby("항목")["금액"].sum().sort_values(ascending=False)
    .rename("지출합계").reset_index()
```

```
)
```

결과는 항목별 합계가 큰 순서의 DataFrame 임. 수입 항목은 지출 순위에서 제외.
지출이 없으면 빈 항목별 표를 반환

4.5 4 단계: 월별 집계

```
monthly = frame.pivot_table(
```

```
    index="월", columns="구분", values="금액",
    aggfunc="sum", fill_value=0
).reindex(columns=["수입", "지출"], fill_value=0).astype("int64")

monthly.columns.name = None
monthly["남은금액"] = monthly["수입"] - monthly["지출"]
monthly["남은비율(%)"] = (
    monthly["남은금액"] / monthly["수입"].replace(0, np.nan) * 100
```

```
).round(1)
```

- pivot_table()로 월별 수입·지출 합계를 구함.
- reindex()로 수입 또는 지출이 전혀 없는 경우에도 두 열이 존재.
- 수입 0 원은 NaN 으로 치환한 후 비율을 계산.
- 반환 전에 reset_index()로 월을 일반 열로 바꿈

빈칸의 의미: 수입이 0 원인 달의 남은비율은 0%가 아니라 계산 불가임. CSV 에서는 빈칸으로 처리. 거래가 전혀 없는 월은 이 코드가 자동 생성하지 않음.

4.6 5 단계: 차트와 목표 비교

항목별 지출 차트에는 categories 를 그대로 사용. 월별 그룹 막대그래프에는 monthly.melt()로 수입·지출 열을 길게 펼친 데이터를 사용.

```
chart = monthly.melt(
    id_vars="월", value_vars=["수입", "지출"],
    var_name="구분", value_name="금액"
)
```

예를 들어 월 | 수입 | 지출 형태를 월 | 구분 | 금액 형태로 바꾸어 색상 기준을 구분에 연결. melt()는 차트용 형태 변환이며 원본 거래를 변경하지 않음.

목표 비교는 별도 목표표를 merge()한 것이 아님. 모든 월에 같은 사이드바 target 값을 넣었음.

```
report = monthly.copy()
report["목표금액"] = int(target)
report["목표와차이"] = report["남은금액"] - report["목표금액"]
```

중요: report 는 화면용 표임. 월별 CSV 는 report 가 아니라 monthly 로 만들었으므로 목표금액과 목표와차이는 다운로드에 포함되지 않음.

---

5. 설정값과 필터의 적용 범위

5.1 설정값별 영향

설정	변경한 결과	변경하지 않은 결과
기간 period	KPI, 항목·월별 집계, 차트, 큰 지출, 상세 조회의 기본 범위, 기간 기준 CSV 3 종	원본 전체 내역, 전체 편집, 전체 백업, 예제 CSV
큰 지출 기준 threshold	표시 열, 큰 지출 표, 조회 내역 CSV 의 표시 값	수입·지출 합계, 항목·월별 합계
월별 목표 target	화면 report 의 목표금액·목표와차이	원본 거래, 집계 결과, 모든 CSV
조회 구분 view_kind	관리 탭의 상세 조회 표 view	KPI·집계·차트·다운로드
조회 항목 view_categories	관리 탭의 상세 조회 표 view	KPI·집계·차트·다운로드

기간·구분·항목을 모두 필터로 부르지만 적용 단계가 다름. “필터가 있다는 사실”보다 **어떤 변수에 필터가 적용되었는지**를 강조하도록 구성.

5.2 다운로드 5 종 비교

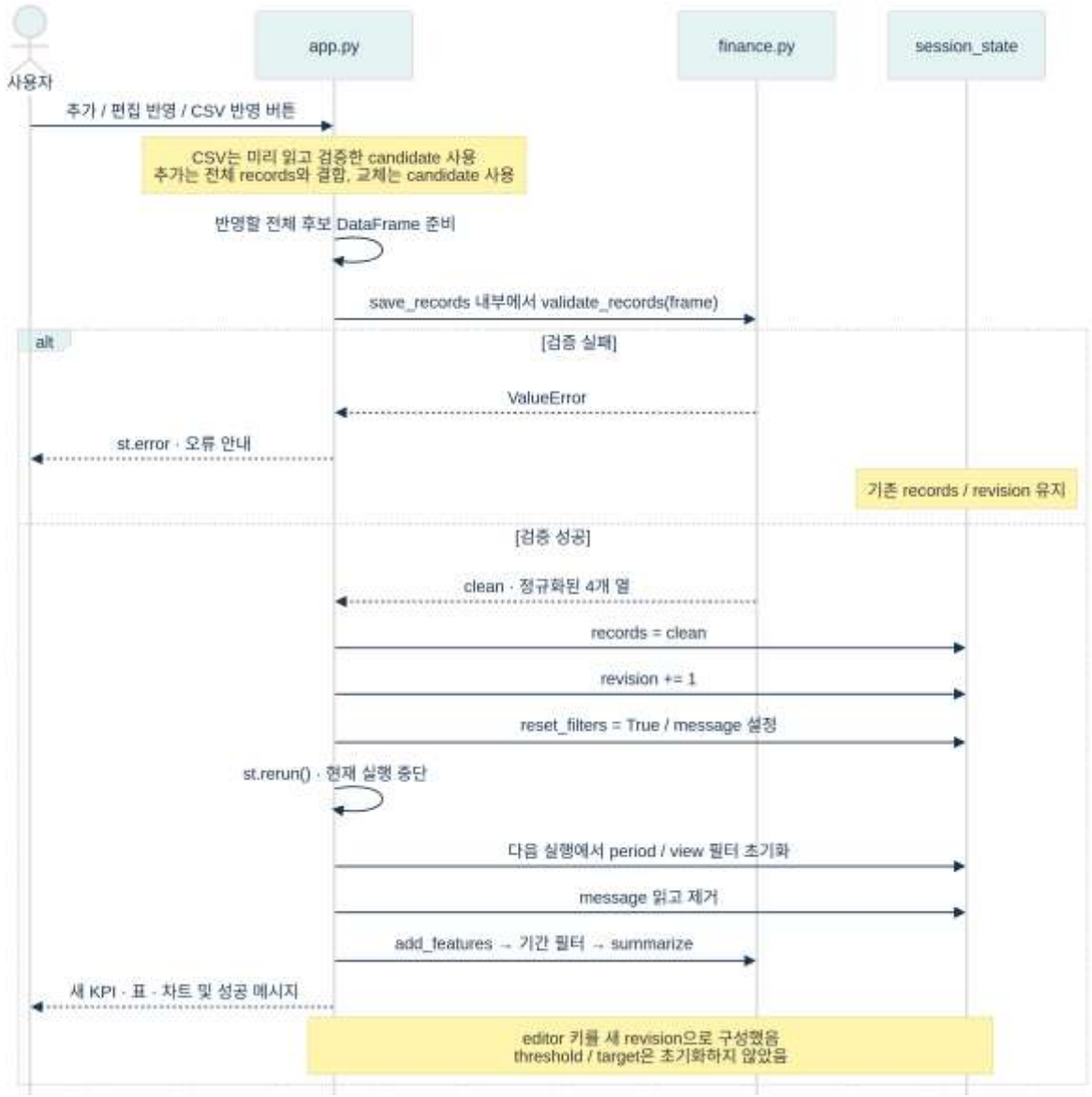
버튼 / 파일	사용 데이터	포함 열	기간 적용
전체 내역 CSV 백업 / 입출금_전체내역.csv	records	날짜, 구분, 항목, 금액	없음

버튼 / 파일	사용 데이터	포함 열	기간 적용
가상 예제 CSV / 입출금_연습용.csv	load_sample()	날짜, 구분, 항목, 금액	없음
조회 내역 CSV / 입출금_조회내역.csv	filtered	날짜, 구분, 항목, 금액, 월, 표시	있음
항목별 지출 CSV / 항목별지출.csv	categories	항목, 지출합계	있음
월별 요약 CSV / 월별요약.csv	monthly	월, 수입, 지출, 남은금액, 남은비율(%)	있음

오해 방지: 상세 조회 표에서 구분·항목을 좁혀도 '조회 내역 CSV'는 view 가 아닌 filtered 를 내보냄. 현재 코드에는 view 그대로를 다운로드하는 버튼이 없음.

---

6. 입력·수정·CSV 반영의 데이터 흐름



6.1 모든 변경을 하나로 반영한 함수

```
def save_records(frame: pd.DataFrame, message: str) -> None:
```

```
    clean = validate_records(frame)
    st.session_state.records = clean
    st.session_state.revision += 1
    st.session_state.reset_filters = True
    st.session_state.message = message
```

```
    st.rerun()
```

save_records()는 app.py 에 정의. 파일이나 DB 에 저장하는 함수가 아니라 **현재 세션의 확정 내역을 갱신하는 함수**

처리 순서는 **후보 전체 표 → 검증 → 세션 반영 → 상태 갱신 → 재실행**임. 검증 결과 clean 이 반환되기 전에 세션 데이터를 바꾸지 않음.

6.2 새 내역 추가

1. 날짜·구분·항목·금액을 폼에서 입력.
2. '내역 추가' 버튼을 누른 시점에 한 행의 DataFrame 을 생성.
3. pd.concat()으로 **전체 확정 내역**과 새 행을 이어 붙였음.
4. 결합된 전체 후보를 save_records()로 전달.

현재 특정 월을 조회하더라도 그 월의 filtered 에만 추가하는 것이 아님. records 전체에 새 행을 추가함. 금액을 입력하는 중간 상태는 확정 내역이 아님.

6.3 전체 내역 수정·삭제

편집기에는 records.copy() 전체를 전달. 화면 편집을 위해 날짜를 datetime.date 로 변환. 수정·추가·삭제한 표는 '수정 내용 반영' 버튼을 눌러야 검증 후 적용

validate_records()가 반영 단계에서 날짜를 다시 YYYY-MM-DD 문자열로 정규화함. 편집기의 자료형과 저장 기준 자료형을 구분.

6.4 CSV 가져오기

업로드 바이트 → 읽기·검증 → candidate 미리보기 → 반영 방식 선택 → 적용 버튼 → 전체 후보 재검증 순서

- 전체 내역 교체: 후보 데이터를 현재 원본으로 대체.
- 기존 내역에 추가: 전체 원본과 후보를 concat().

업로드한 후보가 5,000 행 이하여도 기존 내역과 합친 결과가 한도를 넘을 수 있음. save_records()에서 결합 결과 전체를 다시 검증하므로 이 경우도 반영을 막음.

---

7. 세션 상태와 Streamlit 재실행

7.1 왜 세션 상태가 필요한가?

일반 변수만 사용하면 새 실행에서 값이 다시 만들어짐. st.session_state 는 같은 사용자 세션 안에서 확정 데이터를 다음 실행까지 이어 주는 역할임.

상태 키	역할
records	확정된 전체 거래
revision	반영 시 증가하는 편집기 버전 번호
reset_filters	다음 실행에서 기간·상세 조회 필터를 초기화하라는 요청
message	다음 실행에서 한 번 보여 줄 성공 메시지
period, threshold, target	사이드바 위젯의 값
view_kind, view_categories	상세 조회 표 필터 값

7.2 최초 실행과 재실행

처음에는 records 키가 없으므로 load_sample()로 원본을 초기화. 이후 같은 세션에서는 기존 records 를 사용함.

일반 위젯 조작은 재실행을 유발하며, 폼 내부 입력은 제출 시 함께 전달되도록 구성. save_records()는 저장 성공 직후 st.rerun()을 호출해 현재 실행을 중단하고 위에서부터 다시 실행하도록 함.

7.3 반영 후 다음 실행의 처리

5 reset_filters 를 읽고 제거.

6 period, view_kind, view_categories 키를 제거해 위젯을 기본 선택으로 초기화.

7 message 를 읽고 제거한 뒤 화면에 성공 메시지를 표시.

8 갱신된 원본을 기준으로 파생열·집계·화면을 다시 만들었음.

9 editor_{revision}이라는 새 키로 편집기를 생성.

threshold 와 target 은 위 필터 초기화 목록에 없으므로 유지됨. CSV 업로드 위젯도 이 코드에서 자동 제거하지 않음.

7.4 revision 의 정확한 의미

편집기를 새로운 위젯 키로 만들어 이전 편집 상태가 다음 원본에 다시 적용되는 문제를 줄였음. **거래별 고유 ID 나 전체 입력의 중복 방지 키는 아님.** 같은 CSV 를 다시 적용하거나 같은 거래를 다시 입력하는 것까지 막지 않음.

완전히 동일한 거래는 duplicated()로 경고하되 자동 삭제하지 않았음. 실제 반복 거래일 수 있기 때문임.

7.5 탭과 저장의 한계

st.tabs()는 화면을 나누는 구조이며 선택한 탭만 계산하는 구조가 아님. 현재 코드는 숨겨진 탭의 처리와 다운로드 데이터 생성도 실행 흐름에 포함.

세션 메모리는 영구 저장소가 아님. 새로고침·연결 종료 등으로 초기화될 수 있으므로 전체 CSV 백업이 필요함. 예제 파일은 최초 로딩 외에도 예제 복원과 예제 다운로드에 사용되므로, 반드시 실행 환경에 있어야 함.

---

8. 검증과 CSV 변환 설계

8.1 validate_records()의 검증 단계

순서	처리
1	DataFrame 컬럼 이름 중복과 필수 열 존재 여부를 검사.
2	전체 거래 수가 5,000 행 이하인지 검사.
3	빈 표면 기본 자료형을 갖춘 empty_records()를 반환.
4	필수 4 개 열을 복사하고 문자열 앞뒤 공백을 제거.
5	날짜 형식·실재 여부·연도 범위를 검사.
6	구분과 항목 이름을 검사.
7	금액이 유한한 숫자이며 범위 내 양의 정수인지 검사.
8	오류가 있으면 해당 검사항목의 잘못된 행 번호를 최대 10 개 안내.
9	날짜·금액을 정규화하고 새 DataFrame 을 반환.

errors="coerce"는 잘못된 날짜나 숫자를 결측값으로 바꾸어 검사하기 위한 설정임. 이후 검증에서 반영을 막으므로, 잘못된 값을 자동 삭제하거나 0 으로 저장하는 동작이 아님.

8.2 read_csv_bytes()의 입력 보호

파일 크기를 5 * 1024 * 1024 바이트로 제한. 화면은 이를 5MB 로 안내하며, 계산상 이진 단위 5MiB 기준임.

CSV 를 읽을 때 최대 5,001 행까지만 읽어 5,000 행 초과를 조기에 발견하도록 . 행 수 한도는 읽기 이후 validate_records()가 판단

UTF-8 계열을 먼저 시도하고 **문자 디코딩 실패 시에만** CP949 로 재시도함. CSV 파싱 오류나 거래 검증 오류를 인코딩만 바꾸어 재시도하지 않음.

빈 데이터 구분: 바이트가 전혀 없는 파일은 오류임. 필수 헤더만 있고 거래 행이 없는 CSV 는 검증을 거쳐 빈 거래 표로 사용할 수 있음.

8.3 csv_bytes()의 출력 처리

DataFrame 을 복사하고 문자 열에서 수식 시작으로 해석될 수 있는 값을 완화한 뒤 CSV bytes 로 변환.

- UTF-8 BOM 을 넣어 엑셀 한글 호환성을 높였음.
- 인덱스를 저장하지 않았음.
- 결측값은 빈칸으로 출력.
- 위험한 시작 문자열에는 작은따옴표를 붙이는 완화 처리를 넣었음.
- 금액·비율 같은 숫자 열은 문자 열 처리 대상에서 제외.

수식 실행 위험을 **완화**하는 처리이며, 모든 스프레드시트 환경의 안전을 보증하는 것은 아님. 문자열에 접두 문자가 추가될 수 있으므로 원본 바이트를 그대로 보관하는 백업과도 구분해야 함.

---

9. 첨부 예제 CSV 로 데이터 흐름 추적

> *sample_transactions.csv 의 18 건을 실제 finance.py 함수로 읽고 검증·분류·집계. 아래 표는 추가로 첨부된 교육용 가상 거래 데이터의 결과임. 실제 개인 계좌 자료를 분석한 것은 아님.*

9.1 예제 파일 검증

확인 항목	결과
파일	sample_transactions.csv
파일 크기	606 바이트
인코딩	UTF-8 BOM 포함
필수 열	날짜, 구분, 항목, 금액
거래 수	18 건 — 헤더를 거래로 세지 않았음.
거래 기간	2026-01-01~2026-03-25
수입 / 지출	수입 3 건 / 지출 15 건
결측값 / 완전히 같은 행	0 개 / 0 건
검증 결과	날짜·구분·항목·금액·행 수 한도를 모두 통과.

파일 경로를 사용하는 load_sample()과 업로드 bytes 를 사용하는 read_csv_bytes()가 같은 정규화된 거래 표를 반환하는 것도 확인.

9.2 원본 거래 18 건

날짜	구분	항목	금액
2026-01-01	수입	월급	2,000,000 원
2026-01-05	지출	월세	600,000 원
2026-01-10	지출	식비	280,000 원

날짜	구분	항목	금액
2026-01-15	지출	교통	70,000 원
2026-01-20	지출	쇼핑	150,000 원
2026-01-25	지출	통신	100,000 원
2026-02-01	수입	월급	2,000,000 원
2026-02-05	지출	월세	600,000 원
2026-02-10	지출	식비	320,000 원
2026-02-15	지출	교통	80,000 원
2026-02-20	지출	쇼핑	300,000 원
2026-02-25	지출	통신	120,000 원
2026-03-01	수입	월급	2,000,000 원
2026-03-05	지출	월세	600,000 원
2026-03-10	지출	식비	250,000 원
2026-03-15	지출	교통	60,000 원
2026-03-20	지출	쇼핑	100,000 원
2026-03-25	지출	통신	80,000 원

매월 월급 1 건과 지출 5 건으로 구성. 월별 건수가 같아도 거래 금액이 다르므로 월별 지출과 남은 금액이 달라짐.

9.3 예제 데이터의 단계별 변화

```
sample_transactions.csv
```

```
    ↓ load_sample() → validate_records()
records                  18 행 × 4 열
    ↓ add_features(threshold=100000)
all_records              18 행 × 6 열 — 월·표시 추가
    ↓ 전체 기간 선택
filtered                 18 행 × 6 열
    ├─ KPI               수입·지출·남은금액·거래 건수
    ├─ categories         5 행 × 2 열 — 항목별 지출 합계
    ├─ monthly            3 행 × 5 열 — 월별 집계
    │    └─ report        3 행 × 7 열 — 목표 비교용 화면 표
    └─ 큰 지출 표        11 행 — 지출이면서 10 만 원 이상
```

1 월을 선택하면 filtered 는 6 행이며, 항목별 표는 5 행, 월별 표는 1 행으로 바뀜. records 는 계속 18 행으로 유지됨.

9.4 전체 기간 KPI

지표	결과
수입 합계	6,000,000 원
지출 합계	3,710,000 원
기간 내 남은금액	2,290,000 원
조회 내역	18 건

해석: 기록된 3 개월 동안 수입이 지출보다 2,290,000 원 많았음. 기초 잔액이나 계좌의 모든 입출금을 반영한 실제 잔액이라는 뜻은 아님.

9.5 항목별 지출 순위

순위	항목	지출합계
1	월세	1,800,000 원
2	식비	850,000 원
3	쇼핑	550,000 원
4	통신	300,000 원
5	교통	210,000 원

expenses.groupby('항목')['금액'].sum()으로 같은 항목의 3 개월 금액을 합산. 가장 비싼 거래 한 건이 아니라 항목별 누적 지출을 비교.

9.6 월별 요약과 목표 비교

기본 목표인 월 700,000 원을 적용.

월	수입	지출	남은금액	남은비율	목표금액	목표와차이
2026-01	2,000,000 원	1,200,000 원	800,000 원	40.0%	700,000 원	+100,000 원
2026-02	2,000,000 원	1,420,000 원	580,000 원	29.0%	700,000 원	−120,000 원
2026-03	2,000,000 원	1,090,000 원	910,000 원	45.5%	700,000 원	+210,000 원

1 월과 3 월은 목표 이상을 남겼고, 2 월은 목표보다 120,000 원 부족. 남은금액이 가장 큰 달은 3 월임. 월별 남은금액은 전월 금액을 더한 누적 잔액이 아님.

화면과 CSV 구분: 위 표는 report 의 화면 결과를 설명. 월별요약.csv 에는 목표금액·목표와차이를 제외한 monthly 의 5 개 열만 들어감.

9.7 분류 기준과 경계값

큰 지출 기준을 100,000 원으로 설정했을 때의 결과임.

구분	전체	1 월	2 월	3 월
수입	3 건	1 건	1 건	1 건
큰 지출	11 건	4 건	4 건	3 건
일반 지출	4 건	1 건	1 건	2 건

1 월 통신 100,000 원과 3 월 쇼핑 100,000 원은 >= 조건에 따라 큰 지출에 포함됨. 월급 2,000,000 원은 금액이 크더라도 지출이 아니므로 '수입'으로 표시함.

기준을 200,000 원으로 변경하면 큰 지출은 7 건으로 줄어듦. 1 월 쇼핑 150,000 원의 표시는 일반 지출로 바뀌지만, 수입·지출·항목별 합계는 그대로임.

9.8 필터 변경 시 확인 결과

시연	결과	바뀌지 않은 부분
기간을 1 월로 선택	조회 6 건, 수입 2,000,000 원, 지출 1,200,000 원, 남은금액 800,000 원	전체 원본·전체 편집·전체 백업 18 건
전체 기간에서 상세 조회 항목을 식비로 선택	상세 표에 식비 3 건 표시	KPI 와 조회 CSV 는 전체 기간 18 건 기준
큰 지출 기준을 20 만 원으로 변경	큰 지출 7 건, 조회 CSV 의 표시 열 변경	금액 합계와 항목·월별 집계
월별 목표만 변경	화면의 목표금액·목표와차이 변경	월별 CSV 를 포함한 모든 CSV

각 시연은 원본 예제 18 건을 유지한 상태에서 수행하도록 구성.